ASSIGNMENT 8 REPORT

Application Optimization, Scalability and Reliability

Name: Aman
Roll No.: 241001160007

1. Objective

This assignment enhances the secure multi-client TCP chat application from Assignment 7 by improving scalability, reliability, maintainability and performance through thread-pool based client management, heartbeat monitoring, automatic reconnection, centralized configuration and benchmarking.

2. Software and Tools

- Python 3
- Tkinter
- Socket Programming
- Mininet
- Wireshark
- ThreadPoolExecutor
- JSON
- CSV
- Matplotlib
- psutil

3. System Architecture

The client-server architecture allows multiple GUI clients to communicate with a centralized TCP server. Assignment 8 introduces ThreadPoolExecutor, heartbeat monitoring, automatic reconnection, graceful shutdown, centralized configuration and performance evaluation.

4. Implemented Optimizations

- ThreadPoolExecutor limits worker threads for better scalability.
- Heartbeat (PING) detects dead clients.
- Automatic reconnection restores unexpected disconnections.
- Graceful shutdown safely disconnects clients.
- Configuration moved to config.json.
- Improved exception handling and resource cleanup.

5. Performance Evaluation

Benchmarking was conducted with 5, 8 and 10 concurrent clients. Average message delay, throughput, CPU usage and memory usage were recorded.

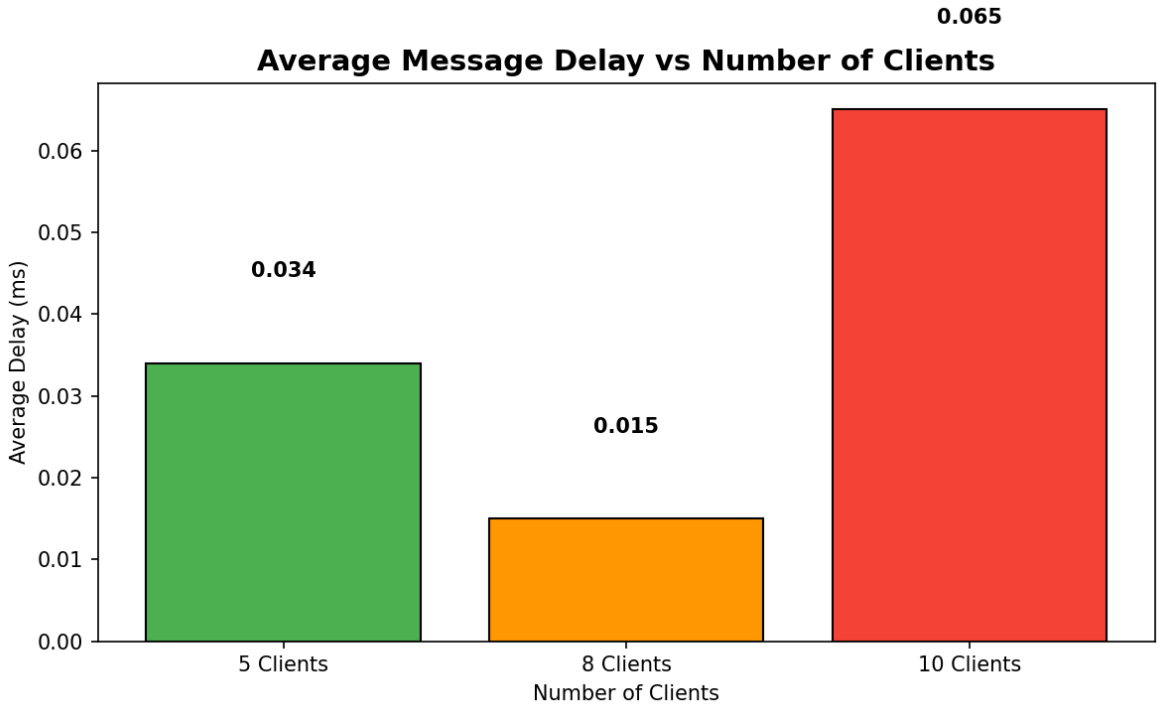


Figure 1: Average Message Delay

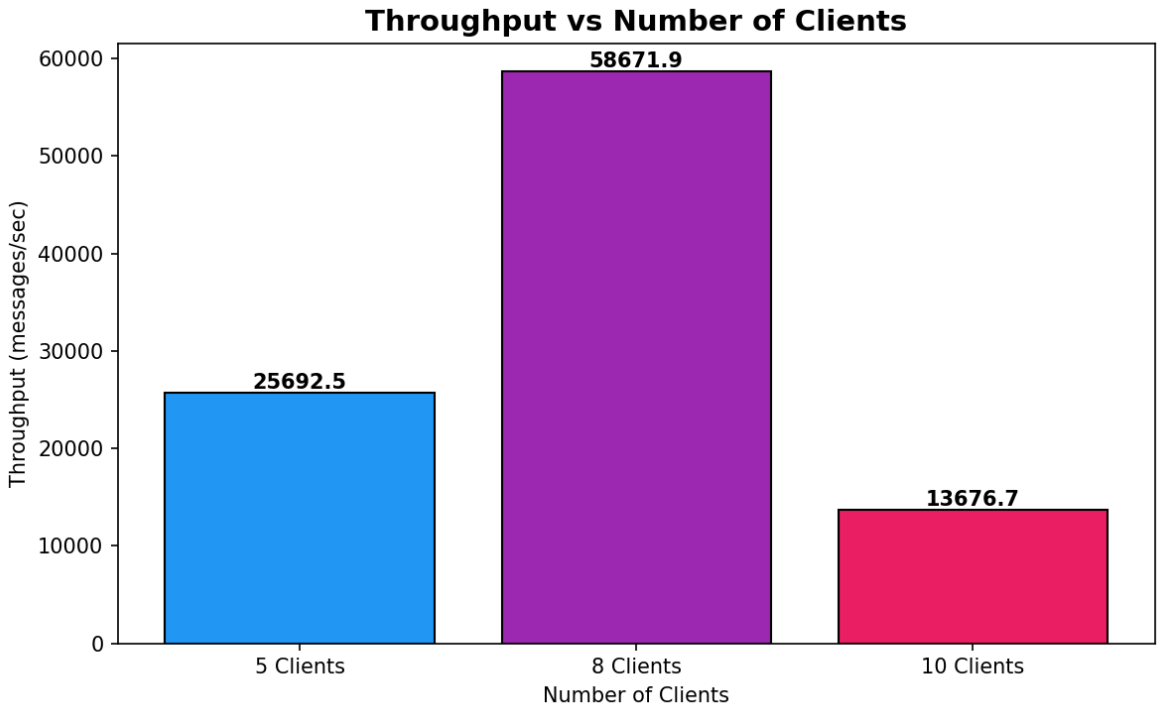


Figure 2: Throughput

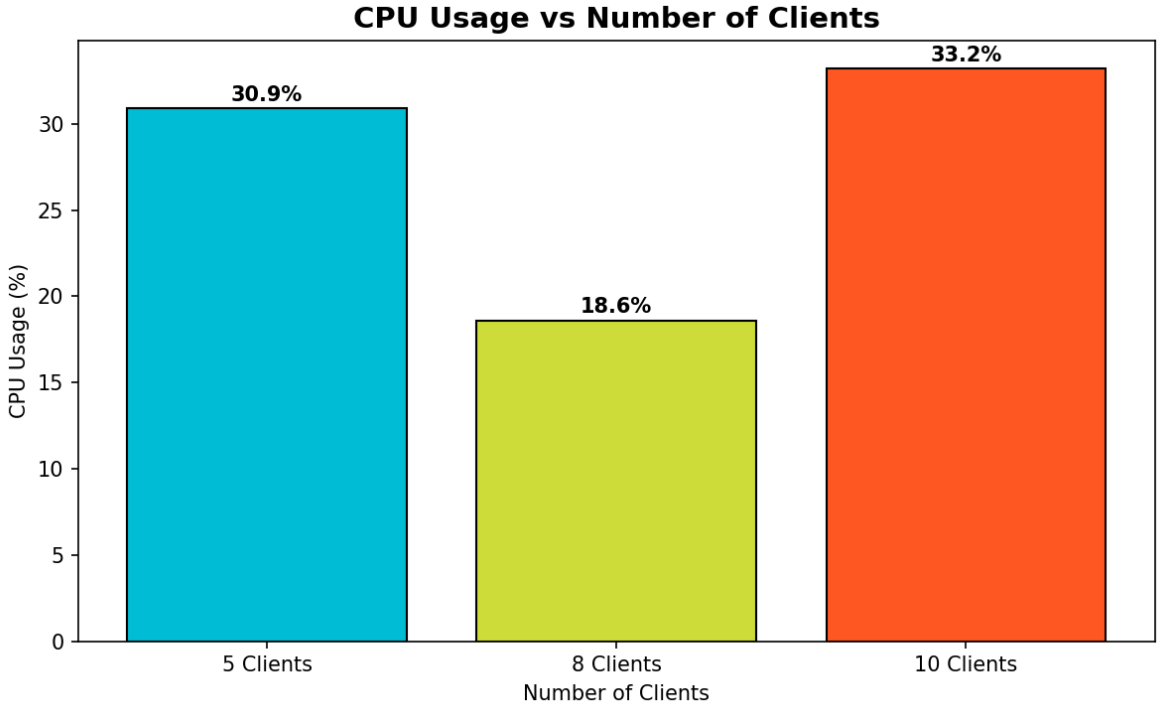


Figure 3: CPU Usage

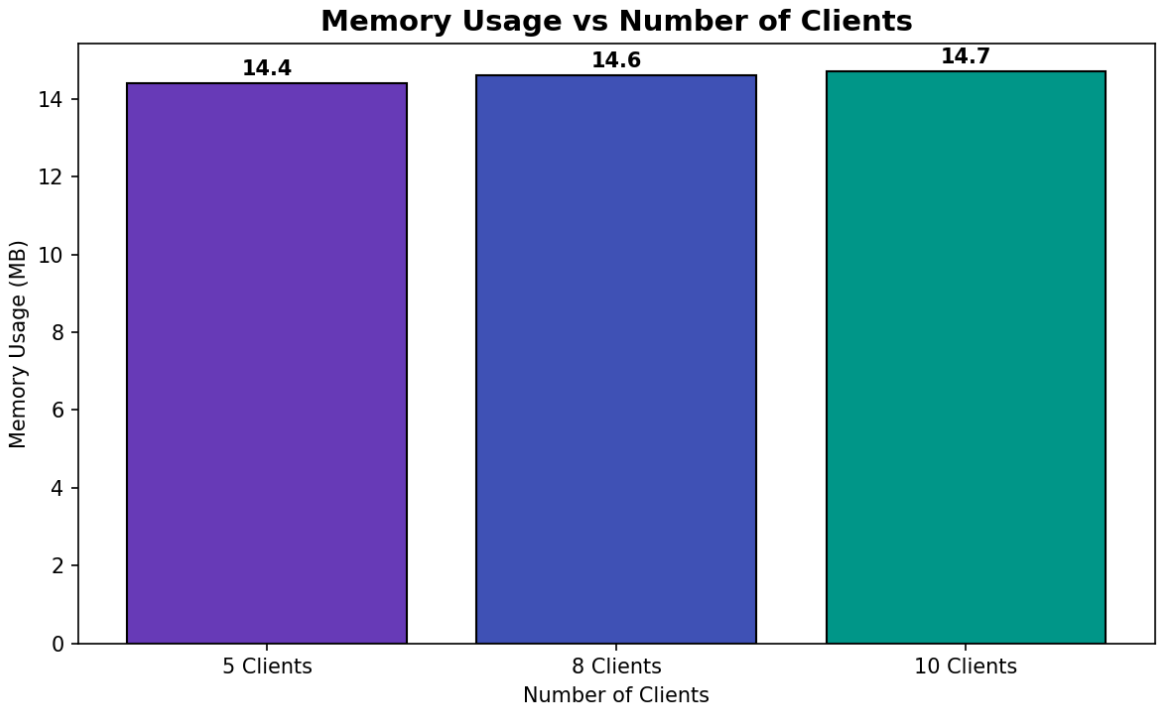


Figure 4: Memory Usage

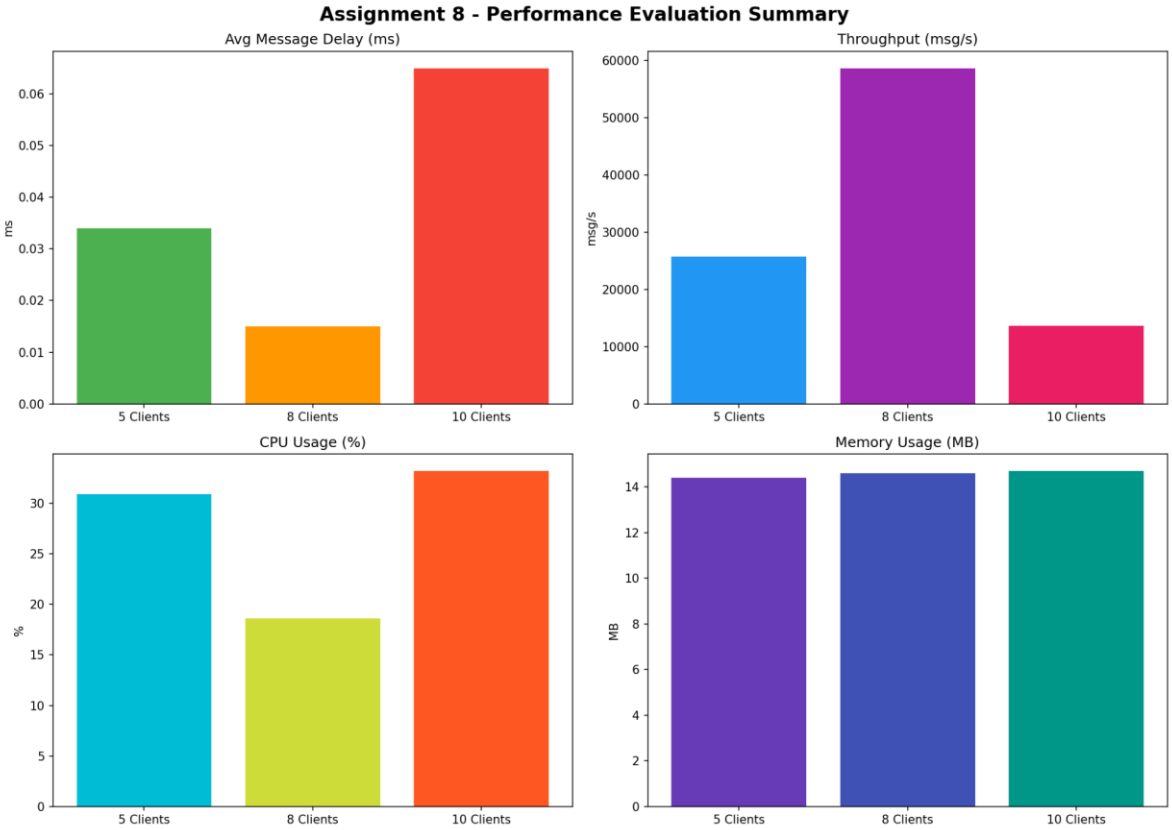


Figure 5: Performance Summary

Performance Analysis

The benchmark demonstrates stable operation with multiple concurrent clients. Memory usage remains nearly constant while CPU utilization increases moderately under heavier workloads. Throughput remains high and message delay stays low, demonstrating that the implemented optimizations improve scalability and reliability.

6. GUI Testing

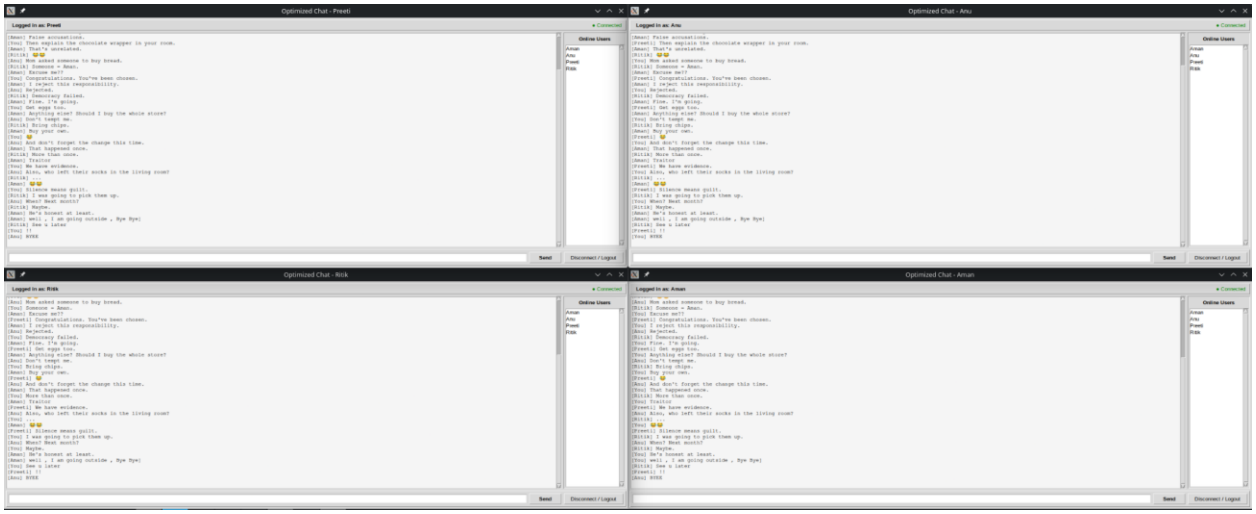


Figure 6: Multi-client Chat Interface

The GUI successfully supports multiple authenticated users communicating simultaneously while displaying online users and chat history.

7. Wireshark Verification

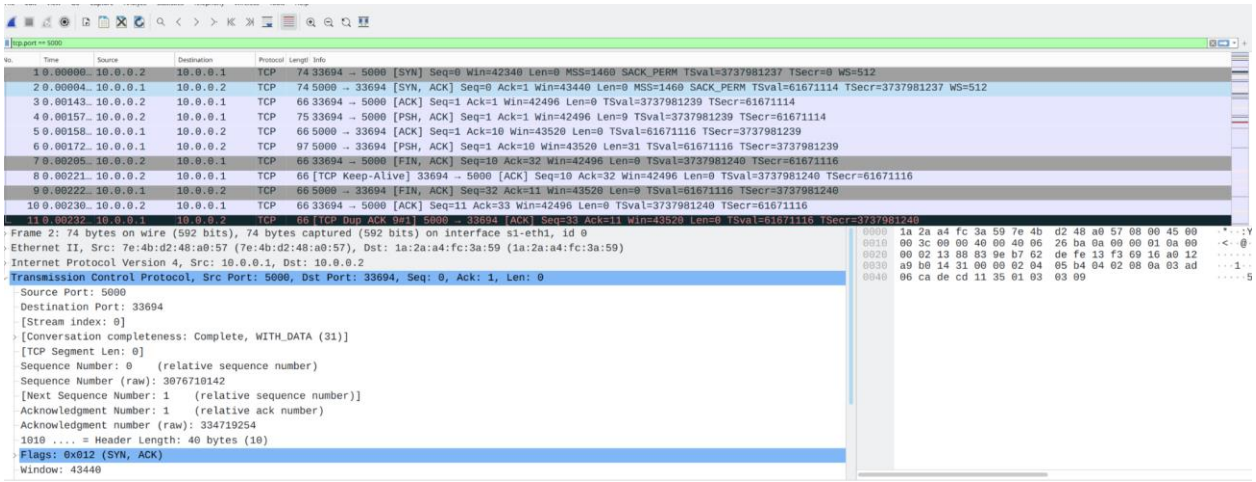


Figure 7: TCP Connection Establishment

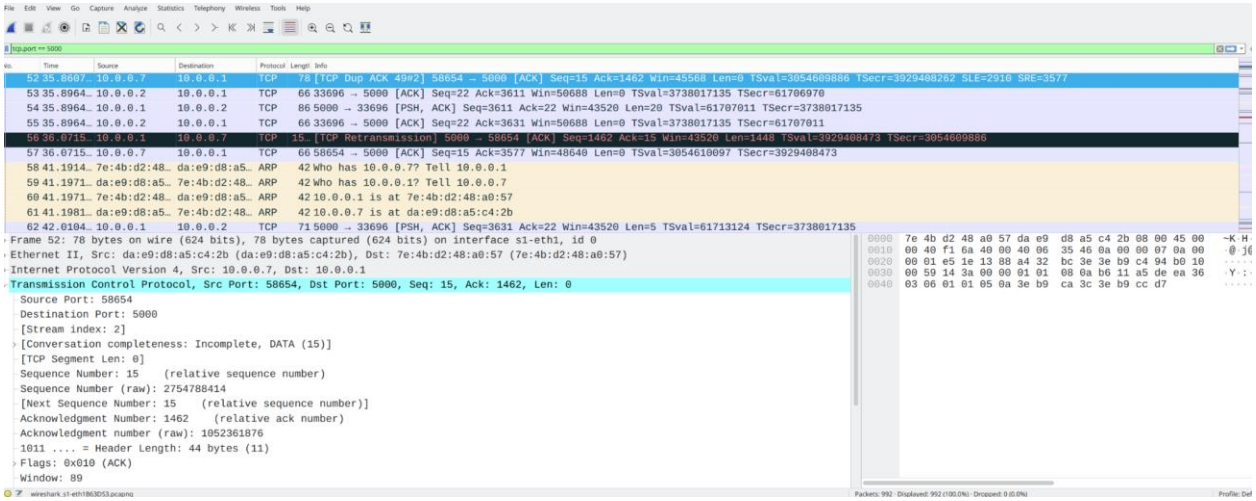


Figure 8: Communication Verification

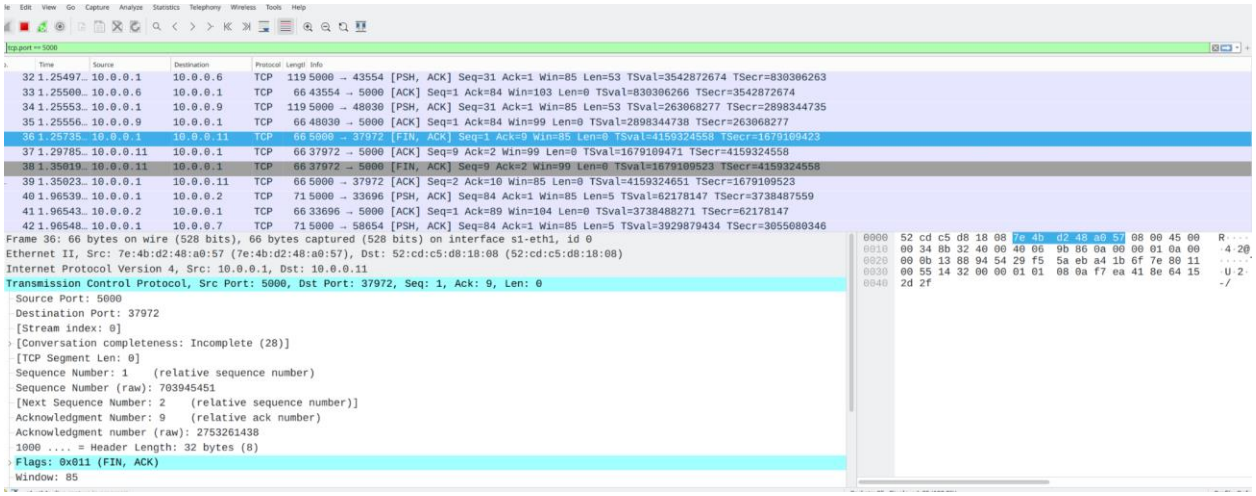


Figure 9: TCP Connection Termination

Wireshark verification confirms the TCP three-way handshake, packet exchange during communication and graceful connection termination.

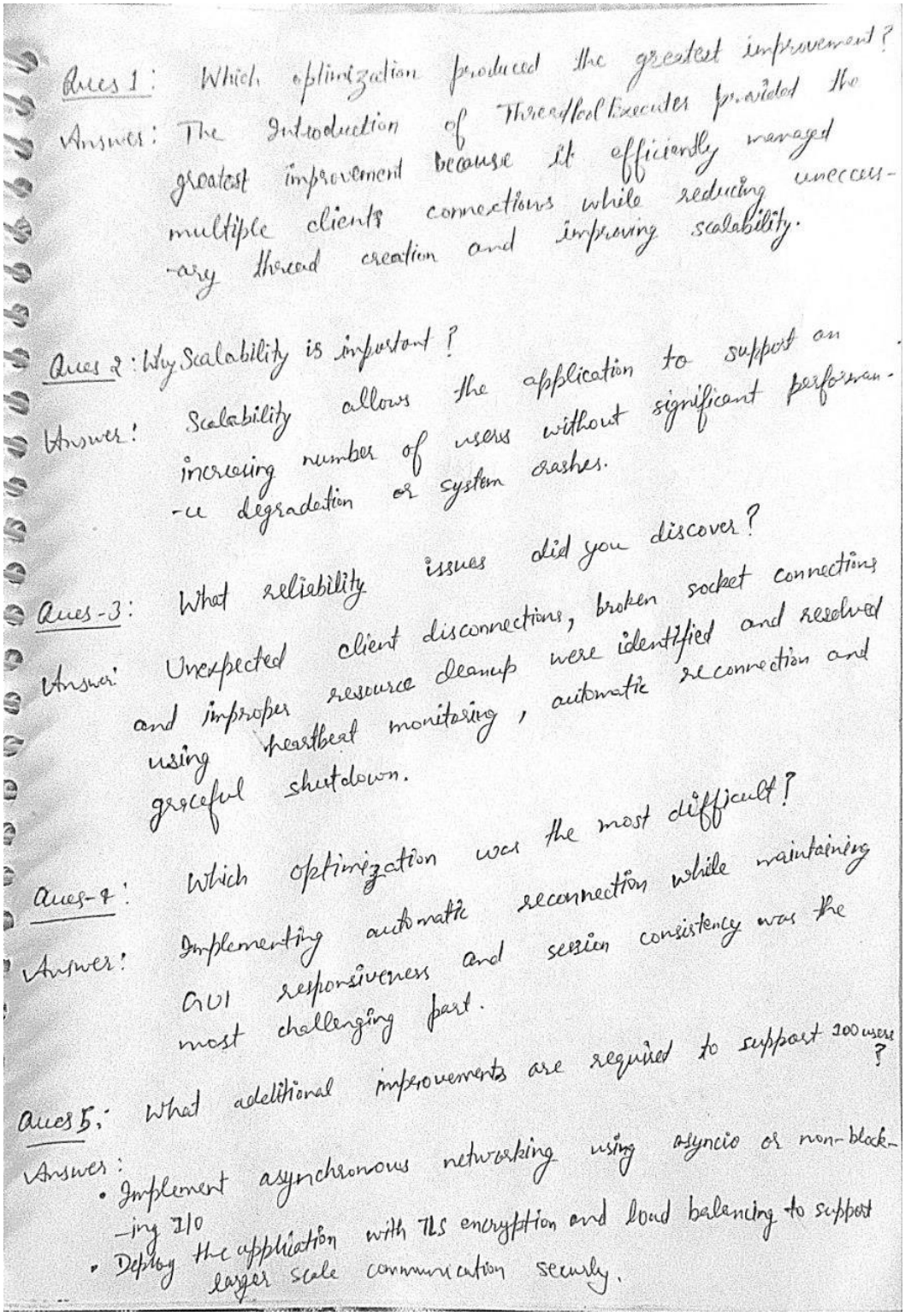
8. Testing Summary

Test	Expected	Status
Authentication	Successful login	Pass
Multiple Clients	10 clients	Pass
Heartbeat	Dead client detection	Pass
Auto Reconnection	Reconnect after disconnect	Pass
Graceful Shutdown	Clean logout	Pass
Benchmark	Performance measured	Pass
Wireshark	TCP verified	Pass

9. Conclusion

Assignment 8 successfully extends the secure chat application by improving scalability, reliability and maintainability. ThreadPoolExecutor, heartbeat monitoring, automatic reconnection, graceful shutdown and centralized configuration enhance system robustness. Performance benchmarking and Wireshark analysis confirm that the optimized application supports multiple concurrent clients while maintaining efficient resource utilization.

Handwritten Reflection answers



Ques 1: Which optimization produced the greatest improvement?

Answer: The introduction of ThreadPoolExecutor provided the greatest improvement because it efficiently managed multiple clients connections while reducing unnecessary thread creation and improving scalability.

Ques 2: Why Scalability is important?

Answer: Scalability allows the application to support an increasing number of users without significant performance degradation or system crashes.

Ques-3: What reliability issues did you discover?

Answer: Unexpected client disconnections, broken socket connections and improper resource cleanup were identified and resolved using heartbeat monitoring, automatic reconnection and graceful shutdown.

Ques-4: Which optimization was the most difficult?

Answer: Implementing automatic reconnection while maintaining GUI responsiveness and session consistency was the most challenging part.

Ques 5: What additional improvements are required to support 200 users?

Answers:
- Implement asynchronous networking using asyncio or non-blocking I/O
- Deploy the application with TLS encryption and load balancing to support larger scale communication securly.